TP-RAG en entreprise - Etapes de mise en place d'un RAG Professionnel.

Ces deux exercices font construire pas à pas un assistant RAG pour la documentation interne d'une entreprise. Le code est fourni sous forme de squelettes à compléter, et chaque étape se termine par une question de compréhension. L'exercice 1 prépare et indexe les documents, l'exercice 2 les interroge, évalue les réponses et publie le projet sur GitHub.

Contexte

L'entreprise fictive Altéa Services veut un assistant qui répond aux questions des salariés à partir de sa documentation interne, en indiquant toujours de quel document vient la réponse. Vous êtes chargés de construire un premier prototype.

Corpus fourni (dossier `docs/`) : 15 documents internes fictifs en PDF et en texte, répartis en trois services.

Service	Exemples de documents
RH	Congés et absences, télétravail, notes de frais, mutuelle
IT	Mot de passe oublié, VPN, demande de matériel, sécurité des e-mails
Général	Règlement intérieur, plan d'accès aux sites, horaires

Outils. Python 3.10 ou plus, `pypdf` (lecture des PDF), `sentence-transformers` (embeddings), `chromadb` (base vectorielle), `ollama` (LLM en local), `streamlit` (interface), Git et un compte GitHub.

```
python -m venv .venv
source .venv/bin/activate      # Windows : .venv\Scripts\activate
pip install pypdf sentence-transformers chromadb openai streamlit
ollama pull llama3.2
```

Les 8 étapes d'un RAG. Un RAG se construit en deux temps : on prépare une fois les documents (exercice 1), puis on les interroge à chaque question (exercice 2).

N°	Étape	Ce qu'elle fait	Exercice
1	Collecte	Lire les fichiers et extraire leur texte	1
2	Nettoyage	Retirer ce qui gêne : espaces, lignes vides, en-têtes répétés	1
3	Découpage (chunking)	Couper le texte en morceaux de taille raisonnable	1
4	Embeddings et indexation	Transformer chaque morceau en vecteur et le ranger dans une base vectorielle	1
5	Recherche	Retrouver les morceaux les plus proches de la question	2
6	Génération	Donner ces morceaux au LLM pour qu'il rédige une réponse sourcée	2
7	Évaluation	Vérifier si les bons documents sont trouvés et si les réponses sont justes	2

N°	Étape	Ce qu'elle fait	Exercice
8	Mise à disposition	Proposer une interface et publier le projet sur GitHub	2

Organisation. Un fichier Python par étape dans un dossier rag_altea/. Les questions marquées ✍ appellent une réponse écrite de quelques lignes dans reponses.md.

Exercise 1 — Preparer et indexer les documents

Objectif : à la fin de l'exercice, tous les documents d'Altéa sont découpés en morceaux et rangés dans une base vectorielle que l'on peut déjà interroger.

Étape	Fichier
1. Collecte	etape1_collecte.py
2. Nettoyage	etape2_nettoyage.py
3. Découpage	etape3_decoupage.py
4. Embeddings et indexation	etape4_indexation.py
Premier dépôt GitHub	—

Les documents sont rangés par service : docs/RH/, docs/IT/, docs/General/.

Étape 1 — Collecte (25 min)

On lit chaque fichier et on garde, pour chaque page, son texte et son origine.

```python
# etape1_collecte.py
from pathlib import Path

from pypdf import PdfReader

def lire_pdf(chemin):
    lecteur = PdfReader(chemin)
    pages = []
    for numero, page in enumerate(lecteur.pages, start=1):
        texte = ...  # TODO : extraire le texte de la page (méthode extract_text)
        pages.append({"source": chemin.name, "page": numero, "texte": texte})
    return pages

def lire_txt(chemin):
    # TODO : lire le fichier en UTF-8 et renvoyer une liste contenant une seule page
    ...

def collecter(dossier="docs"):
    pages = []
    for chemin in sorted(Path(dossier).rglob("*")):
        service = chemin.parent.name           # RH, IT ou General
        # TODO : selon l'extension (.pdf ou .txt), appeler la bonne fonction,
        #        ajouter la clé "service" à chaque page, puis l'ajouter à la liste
        ...
    return pages

if __name__ == "__main__":

pages = collecter()
print(len(pages), "pages lues")
```

Q1. Complétez le code et affichez le nombre de pages lues pour chaque document.

Q2 ✍ Pourquoi conserve-t-on le nom du fichier, la page et le service, et pas seulement le texte ?
Donnez deux usages concrets dans l'assistant final.

Étape 2 - Nettoyage (15 min)

Le texte extrait d'un PDF contient souvent des espaces en trop, des lignes vides et des mots coupés en fin de ligne.

```python
# etape2_nettoyage.py
import re

def nettoyer(texte):
    texte = re.sub(r"[ \t]+", " ", texte)       # plusieurs espaces -> un seul
    # TODO : remplacer 3 sauts de ligne ou plus par 2 (motif r"\n{3,}")
    # TODO : recoller les mots coupés en fin de ligne (motif r"-\n(\w)" remplacé par r"\1")
    return texte.strip()

```

Q3. Appliquez `nettoyer` à toutes les pages et supprimez les pages devenues vides.

Q4 ✍Affichez une même page avant et après nettoyage.
Citez un défaut qui reste (en-tête répété, numéro de page, tableau mal lu…) et proposez une règle pour le corriger.

Étape 3 - Découpage

Un document entier est trop long pour être envoyé au LLM : on le coupe en morceaux (chunks) qui se chevauchent un peu, pour ne pas perdre une phrase coupée en deux.

```python
# etape3_decoupage.py
def decouper(texte, taille=500, chevauchement=50):
    assert chevauchement < taille, "le chevauchement
        doit être plus petit que la taille"
    morceaux = []
    debut = 0
    while debut < len(texte):
        fin = ...          # TODO : fin du morceau
        morceaux.append(texte[debut:fin])
        if fin >= len(texte):
            break
        debut = ...    # TODO : avancer en gardant le chevauchement
    return morceaux


def decouper_pages(pages, taille=500, chevauchement=50):
    chunks = []
    for p in pages:
        for i, morceau in enumerate(decouper(p["texte"], taille, chevauchement)):
            chunks.append({
                "id": f"{p['source']}-p{p['page']}-{i}",
                "texte": morceau,
                "source": p["source"],
                "page": p["page"],
                "service": p["service"],
            })
    return chunks

```

Q5. Complétez `decouper`. Vérifiez votre code : `decouper("abcdefghij", taille=4, chevauchement=1)` doit renvoyer ["abcd", "defg", "ghij"]. Expliquez à la main pourquoi.

Q6. Découpez tout le corpus avec trois tailles et complétez le tableau:

Taille (caractères)	Chevauchement	Nombre de chunks	Longueur moyenne

Taille (caractères)	Chevauchement	Nombre de chunks	Longueur moyenne
200	20		
500	50		
1000	100		

Q7 ✍ Quel est le risque d'un chunk trop petit?
D'un chunk trop grand?
À quoi sert le chevauchement ?
Quelle taille retenez-vous pour la suite ?

Étape 4 - Embeddings et indexation (40 min)

Un modèle d'embedding transforme un texte en une liste de nombres (un vecteur). Deux textes de sens proche ont des vecteurs proches.

```python
# etape4_indexation.py — partie A : découvrir les embeddings
from sentence_transformers import SentenceTransformer, util

modele = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

phrases = [
    "Comment poser un jour de congé ?",
    "Quelle est la procédure pour demander des vacances ?",
    "Mon ordinateur ne se connecte pas au VPN.",
]
vecteurs = modele.encode(phrases)
print(vecteurs.shape)
print(util.cos_sim(vecteurs, vecteurs))

```

Q8. Quelle est la dimension d'un vecteur ? Combien de nombres faut-il stocker pour tout le corpus avec la taille de chunk retenue ?

Q9 ✍Lisez la matrice de similarité. Quelles phrases sont proches ?
Pourquoi les phrases 1 et 2 le sont-elles alors qu'elles n'ont presque aucun mot en commun ?

```python
# etape5_indexation.py — partie B : ranger les chunks dans la base vectorielle
import chromadb

client = chromadb.PersistentClient(path="base_vectorielle")
collection = client.get_or_create_collection("altea",
                metadata={"hnsw:space": "cosine"})

textes = [c["texte"] for c in chunks]     # chunks issus de l'étape 3
vecteurs = modele.encode(textes, show_progress_bar=True).tolist()

collection.upsert(
    ids=[c["id"] for c in chunks],
    documents=textes,
    embeddings=vecteurs,
    metadatas=...,   # TODO : une liste de dictionnaires {"source", "page", "service"}
)
print(collection.count(), "chunks indexés")

```

Q10. Complétez la partie B, en enchaînant les étapes 1 à 4 dans un même script, et vérifiez que le nombre de chunks indexés est égal au nombre de chunks produits.

Q11. Faites une première recherche pour 3 questions de salariés et affichez, pour chacune, les 3 chunks les plus proches avec leur source:

```
resultat = collection
    .query(query_embeddings=modele
    .encode(["Combien de jours de télétravail ?"])
    .tolist(), n_results=3)
print(resultat["documents"], resultat["metadatas"],
      resultat["distances"])
```

Q12 ✍Relancez le script d'indexation une deuxième fois. Le nombre de chunks double-t-il ? Pourquoi utilise-t-on `upsert` plutôt que `add` ? Que faudra-t-il faire le jour où un document de l'entreprise est modifié ?

Premier dépôt GitHub

```
git init -b main
printf ".venv/\n__pycache__/\nbase_vectorielle/\n.env\n" > .gitignore
pip freeze > requirements.txt
git add .
git commit -m "Exercice 1 : collecte, nettoyage, découpage et indexation"
git remote add origin https://github.com/<votre-compte>/rag-altea.git
git push -u origin main
```

Q13 ✍Pourquoi le dossier `base_vectorielle/` n'est-il pas envoyé sur GitHub ? Comment un collègue qui clone le dépôt pourra-t-il la recréer ?

Exercice 2 - Interroger, évaluer et livrer (2 h)

Objectif : à la fin de l'exercice, un salarié pose une question dans une page web et obtient une réponse avec ses sources ; vous savez dire, chiffres à l'appui, si l'assistant se trompe ; le projet est publié sur GitHub. On repart de la base vectorielle de l'exercice 1.

Étape	Fichier
5. Recherche	etape5_recherche.py
6. Génération	etape6_generation.py
7. Évaluation	etape7_evaluation.py
8. Mise à disposition et GitHub	app.py, README.md

Étape 5 - Recherche (25 min)

On transforme la question en vecteur, puis on demande à la base les chunks les plus proches.

```python
# etape6_recherche.py
import chromadb
from sentence_transformers import SentenceTransformer

modele = SentenceTransformer(
    "paraphrase-multilingual-MiniLM-L12-v2")
collection = chromadb.PersistentClient(
        path="base_vectorielle").get_collection("altea")

def rechercher(question, k=3, service=None):
    vecteur = modele.encode([question]).tolist()
    filtre = {"service": service} if service else None
    res = collection.query(query_embeddings=vecteur,
                n_results=k, where=filtre)
    resultats = []
    for texte, meta, distance in zip(res["documents"][0],
            res["metadatas"][0], res["distances"][0]):
        # TODO : ajouter un dictionnaire avec texte,
        # source, page, service
        # et score = 1 - distance (plus le score
        # est proche de 1, plus c'est proche)
        ...
    return resultats

```

Q1. Complétez `rechercher` et affichez les résultats (source, page, score) pour 3 questions de salariés.

Q2 ✍ Essayez k = 1, 3 et 10 sur une même question. Que gagne-t-on et que perd-on quand k augmente ?

Q3 ✍Relancez une question RH avec `service="RH"`, puis avec `service="IT"`. Qu'observez-vous? En quoi ce filtre est-il une première étape vers la confidentialité en entreprise, et pourquoi ne suffit-il pas ?

```python
# etape7_generation.py
from openai import OpenAI

from etape5_recherche import rechercher

client = OpenAI(base_url="http://localhost:11434/v1",
        api_key="ollama") # Ollama en local
MODELE = "llama3.2"
REPLI = "Je ne trouve pas cette
        information dans la documentation."

def construire_prompt(question, extraits):
    contexte = ""
    for i, e in enumerate(extraits, start=1):
        contexte += ...   # TODO : "[i] (source, page)"
                          #puis le texte, puis une ligne vide
```

```
    return f"""Tu es l'assistant
    interne d'Altéa Services.
    Réponds à la question en utilisant
    UNIQUEMENT les extraits ci-dessous.
    Après chaque information, indique
    le numéro de l'extrait entre
    crochets, par exemple [1].
    Si les extraits ne contiennent pas
    la réponse, réponds exactement : "{REPLI}"
Extraits :
{contexte}
Question : {question}
Réponse :"""

def repondre(question, k=3):
    extraits = rechercher(question, k)
    prompt = construire_prompt(question, extraits)
    reponse = client.chat.completions.create(
        model=MODELE,
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return reponse.choices[0].message.content, extraits
```

Q4. Complétez construire_prompt, puis affichez le prompt complet envoyé pour une question. Repérez ses trois parties : consigne, extraits, question.

Q5. Posez 5 questions, dont 2 sans réponse dans la documentation (par exemple « Quel temps fera-t-il demain ? »). Complétez :

Question	Réponse juste ?	Sources citées ?	Phrase de repli quand il le fallait ?

Q6 ✍Posez directement au LLM, sans extraits, la question « Combien de jours de télétravail par semaine sont autorisés chez Altéa ? ». Comparez avec la réponse du RAG. Qu'est-ce qu'une hallucination, et comment le RAG la limite-t-il ?

Q7 ✍ Pourquoi fixe-t-on temperature=0 pour un assistant documentaire ?

Étape 7 - Évaluation (30 min)

Un assistant d'entreprise doit être mesuré avant d'être déployé. On vérifie d'abord la recherche, puis les réponses.

Q8. Créez evaluation.csv avec 10 questions et le document qui contient la réponse :

```
question,document_attendu
Combien de jours de télétravail par semaine ?,teletravail.pdf
Comment réinitialiser mon mot de passe ?,mot_de_passe.txt
```

Q9. Complétez le calcul du taux de réussite de la recherche :

```python
# etape8_evaluation.py
import csv
from etape5_recherche import rechercher

def taux_reussite(fichier="evaluation.csv", k=3):
    with open(fichier, encoding="utf-8") as f:
        lignes = list(csv.DictReader(f))
    trouves = 0
    for ligne in lignes:
        sources = [r["source"] for r in rechercher(ligne["question"], k)]
        if ...:   # TODO : le document attendu
                  #fait-il partie des sources trouvées ?
            trouves += 1
    return trouves / len(lignes)


for k in (1, 3, 5):
    print(f"k = {k} : {taux_reussite(k=k):.0%}
                de questions où le bon document est trouvé")
```

Q10. Notez vous-mêmes les 10 réponses générées : juste, partielle ou fausse, et sources exactes ou non. Donnez le pourcentage de réponses justes et de réponses correctement sourcées.

Q11 ✍ Choisissez une réponse fausse. Le bon document avait-il été trouvé à l'étape 5 ? Si non, l'erreur vient de la recherche ; si oui, elle vient de la génération. Proposez une amélioration (taille des chunks, valeur de k, consigne du prompt…), testez-la et donnez le nouveau résultat.

Étape 8 - Mise à disposition et GitHub (30 min)

```python
#app.py
import streamlit as st

from etape6_generation import repondre

st.title("Assistant documentaire Altéa")
question = st.text_input("Votre question")

if question:
    with st.spinner("Recherche dans la documentation…"):
        reponse, extraits = repondre(question)
    st.write(reponse)
    st.subheader("Sources")
    for i, e in enumerate(extraits, start=1):
        # TODO : un st.expander titré "[i]
        #source, page X" qui affiche le texte de l'extrait
        ...
```

Q12. Complétez app.py et lancez-le avec streamlit run app.py. Faites une capture d'écran d'une réponse avec ses sources.

Q13. Rédigez le README.md : objectif du projet, installation, ordre de lancement des scripts, exemple de question, résultats de l'étape 7, limites connues, auteurs. Mettez à jour requirements.txt, puis publiez :

```
git add .
git commit -m "Exercice 2 : recherche, génération, évaluation et interface"
git push
git tag v1.0 && git push origin v1.0
```

Q14 ✍Votre prototype fonctionne. Citez quatre points à traiter avant de le proposer à tous les salariés d'Altéa (par exemple : droits d'accès selon le salarié, données personnelles dans les documents, mise à jour des documents, coût et hébergement du LLM), en expliquant chacun en deux lignes.